AI ASSISTED CODING

ASSIGNMENT- 17.1

NAME: G.V.N Hasini

HTNO: 2303A51099

Task 1: Perform EDA on Penguin dataset(from seaborn)

• Load and display entire dataset

```
import seaborn as sns
import pandas as pd

# Load the penguin dataset
penguins_df = sns.load_dataset('penguins')

# Display the first 5 rows of the DataFrame
display(penguins_df.head())
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

• Display country wise count

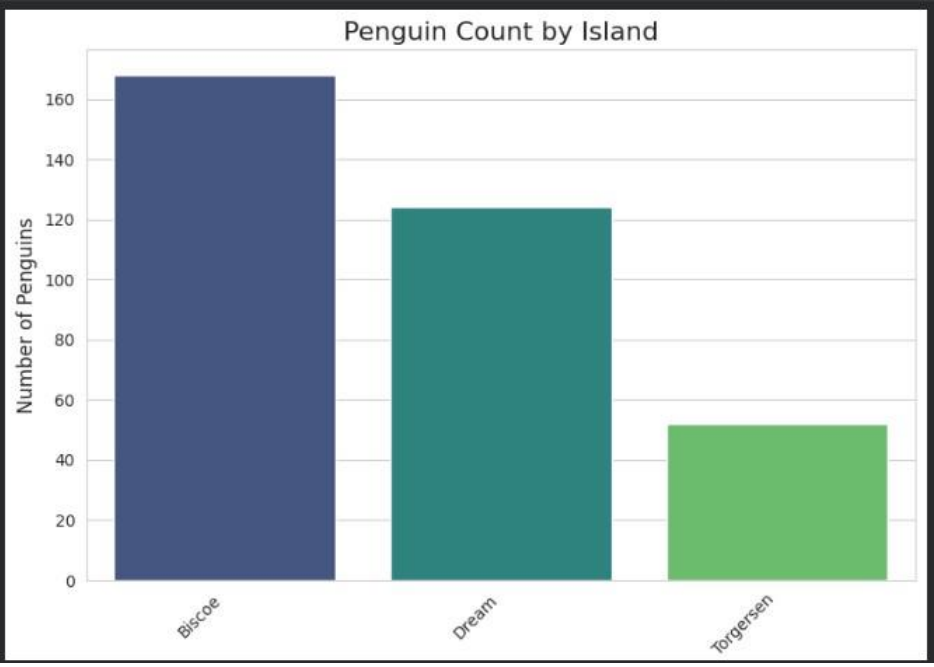
```
# Calculate the count of penguins by island
island_counts = penguins_df['island'].value_counts()

# Display the island counts
display(island_counts)
```

island	count
Biscoe	168
Dream	124
Torgersen	52

dtype: int64

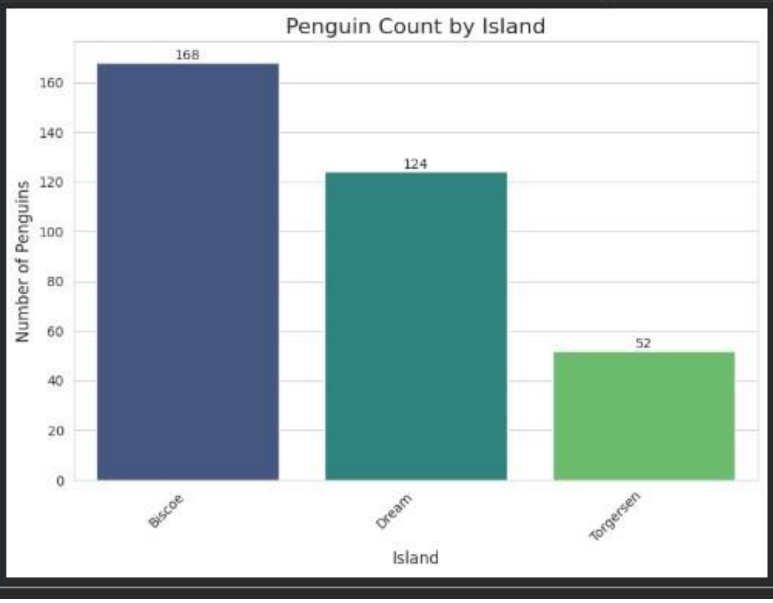
• Visualize bar chart (country wise)

```python
# Set a style for the plots for better aesthetics
sns.set_style("whitegrid")

# Create the bar plot
plt.figure(figsize=(8, 6))
sns.barplot(x=island_counts.index, y=island_counts.values, hue=island_counts.index, palette='viridis', legend=False)

# Add titles and labels
plt.title('Penguin Count by Island', fontsize=16)
plt.xlabel('Island', fontsize=12)
plt.ylabel('Number of Penguins', fontsize=12)

# Rotate x-axis labels if needed
plt.xticks(rotation=45, ha='right')

# Display the plot
plt.tight_layout()
plt.show()
```



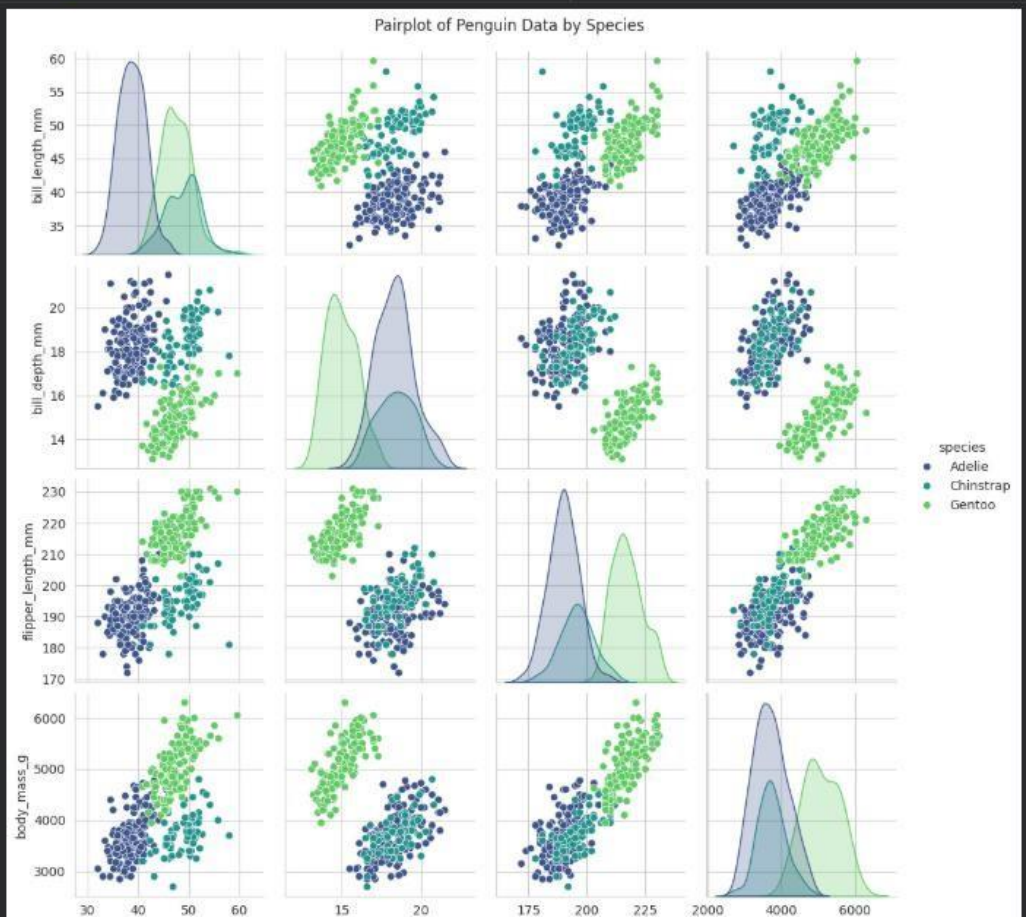
- Visualize bar chart with values count at top

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Set a style for the plots for better aesthetics
sns.set_style("whitegrid")

# Create the bar plot
plt.figure(figsize=(8, 6))
ax = sns.barplot(x=island_counts.index, y=island_counts.values, hue=island_counts.index, palette='viridis', legend=False)

# Add titles and labels
plt.title('Penguin Count by Island', fontsize=16)
plt.xlabel('Island', fontsize=12)
plt.ylabel('Number of Penguins', fontsize=12)

# Add value counts on top of each bar
for container in ax.containers:
    ax.bar_label(container, fmt='%d')

# Rotate x-axis labels if needed
plt.xticks(rotation=45, ha='right')

# Display the plot
plt.tight_layout()
plt.show()
```

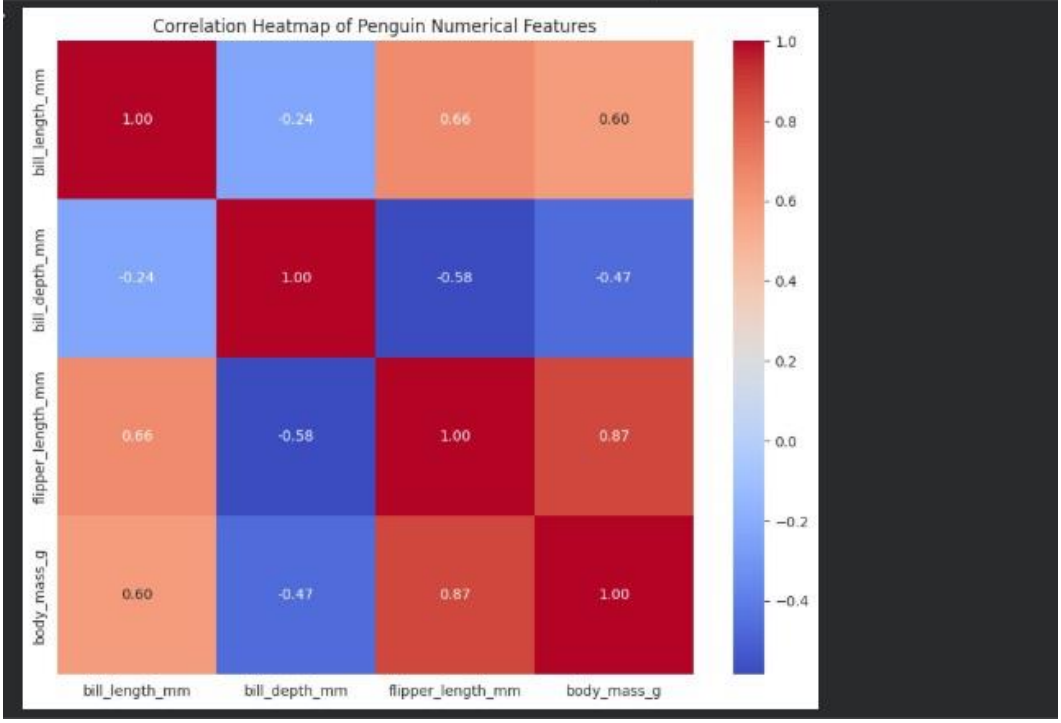


- Visualize pairplot and other plots

```python
import seaborn as sns
import matplotlib.pyplot as plt

# Create a pairplot for the penguins_df
sns.pairplot(penguins_df, hue='species', palette='viridis')
plt.suptitle('Pairplot of Penguin Data by Species', y=1.02) # Add a title above the plots
plt.show()
```



```
# Calculate the correlation matrix for numerical features
correlation_matrix = penguins_df.select_dtypes(include=['float64', 'int64']).corr()

# Create a heatmap of the correlation matrix
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap of Penguin Numerical Features')
plt.show()
```



Task 2: Perform EDA on HealthExp dataset(seaborn)

• Load and display entire dataset

```
import seaborn as sns
import pandas as pd

# Load the HealthExp dataset
healthexp_df = sns.load_dataset('healthexp')

# Display the entire DataFrame for initial inspection
display(healthexp_df.head())
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

• Display columns and statistical summary (describe)

```python
# Display the columns of the DataFrame
print("Columns in healthexp_df:")
display(healthexp_df.columns)

# Display the statistical summary of the DataFrame
print("\nStatistical summary of healthexp_df:")
display(healthexp_df.describe())
```

```
Columns in healthexp_df:
Index(['Year', 'Country', 'Spending_USD', 'Life_Expectancy'], dtype='object')

Statistical summary of healthexp_df:
```

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

• Display missing values , and draw bar chart

```python
# Display missing values
missing_values = healthexp_df.isnull().sum()
print("Missing values in each column:")
display(missing_values[missing_values > 0])

# Filter out columns with no missing values for the bar chart
missing_values_to_plot = missing_values[missing_values > 0]

# Draw a bar chart for missing values if there are any
if not missing_values_to_plot.empty:
    plt.figure(figsize=(10, 6))
    sns.barplot(x=missing_values_to_plot.index, y=missing_values_to_plot.values, palette='viridis')
    plt.title('Number of Missing Values Per Column')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print("No missing values found in the dataset.")
```

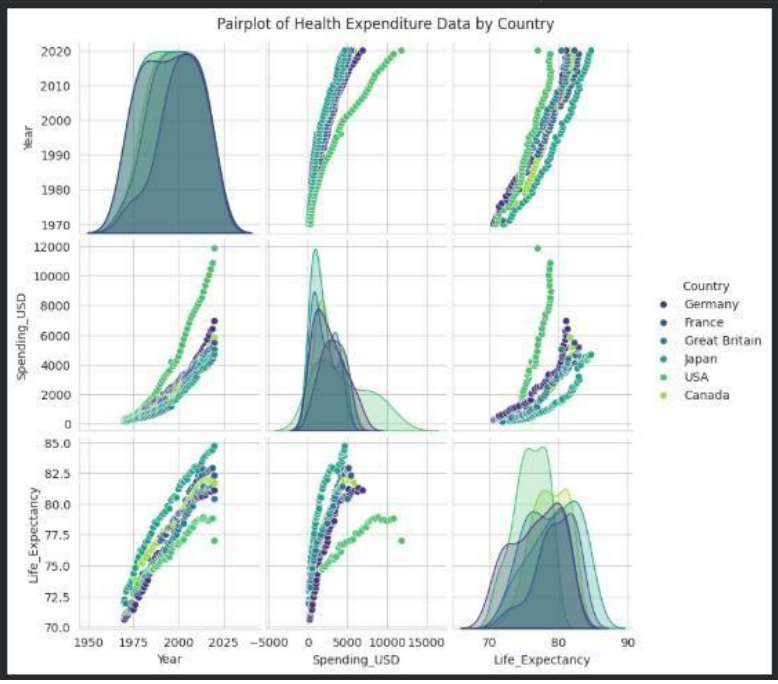
```
Missing values in each column:
```

	0

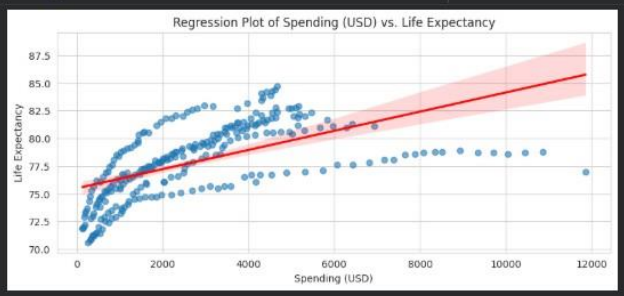
```
dtype: int64
No missing values found in the dataset.
```

• Visualize pairplot, regplot and box plot, scatter plot etc

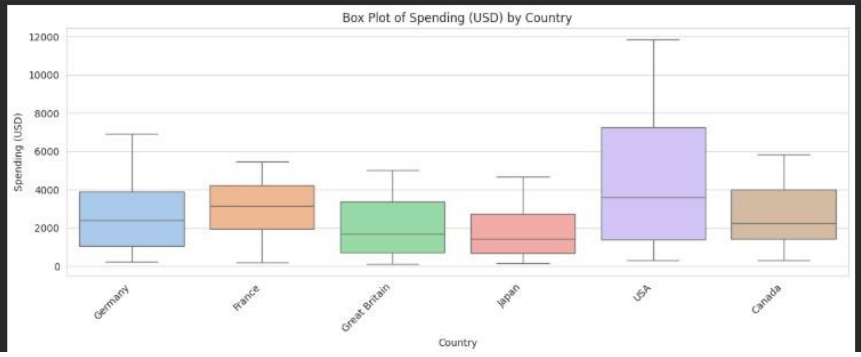
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Create a pairplot of the healthexp_df, colored by 'Country'
sns.pairplot(healthexp_df, hue='Country', palette='viridis')
plt.suptitle('Pairplot of Health Expenditure Data by Country', y=1.02) # Add a title above the plots
plt.show()
```




```python
# Create a regression plot for Spending_USD vs Life_Expectancy
plt.figure(figsize=(10, 4))
sns.regplot(x='Spending_USD', y='Life_Expectancy', data=healthexp_df, scatter_kws={'alpha':0.6}, line_kws={'color':'red'})
plt.title('Regression Plot of Spending (USD) vs. Life Expectancy')
plt.xlabel('Spending (USD)')
plt.ylabel('Life Expectancy')
plt.grid(True)
plt.show()
```




```python
# Create a box plot of Spending_USD by Country
plt.figure(figsize=(12, 5))
sns.boxplot(x='Country', y='Spending_USD', data=healthexp_df, hue='Country', palette='pastel', legend=False)
plt.title('Box Plot of Spending (USD) by Country')
plt.xlabel('Country')
plt.ylabel('Spending (USD)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Task 3: Perform following task on given dataset (Train the model)

• Analyze the Planets dataset to understand its structure, features, and missing values.

• Perform data cleaning and preprocessing to handle missing or incorrect data.

• Train the dataset using different machine learning models to make predictions.

• Compare and explain the performance of all trained models with proper reasoning.

```python
import seaborn as sns
import pandas as pd

# Load the planets dataset
planets_df = sns.load_dataset('planets')

# Display the first 5 rows of the DataFrame
display(planets_df.head())

# Display information about the DataFrame (columns, non-null counts, dtypes)
print("\nDataFrame Info:")
planets_df.info()
```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

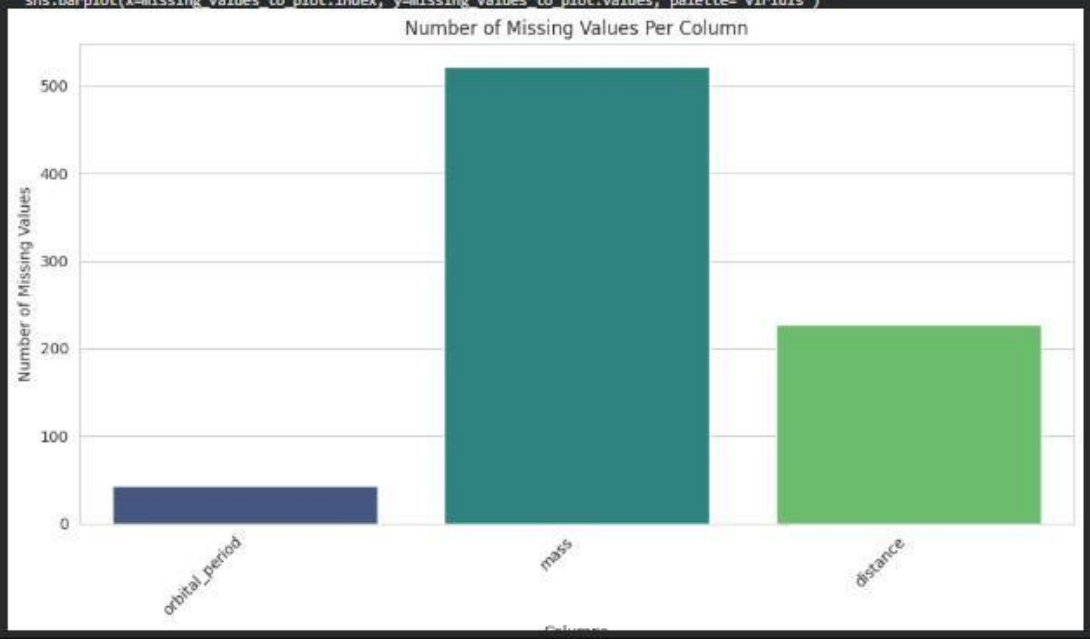
```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1035 entries, 0 to 1034
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   method          1035 non-null   object
 1   number          1035 non-null   int64
 2   orbital_period  992 non-null    float64
 3   mass            513 non-null    float64
 4   distance        808 non-null    float64
 5   year            1035 non-null   int64
dtypes: float64(3), int64(2), object(1)
memory usage: 48.6+ KB
```

```
import matplotlib.pyplot as plt

# Display missing values count for each column
missing_values = planets_df.isnull().sum()
print("\nMissing values in each column:")
display(missing_values[missing_values > 0])

# Filter out columns with no missing values for the bar chart
missing_values_to_plot = missing_values[missing_values > 0]

# Draw a bar chart for missing values if there are any
if not missing_values_to_plot.empty:
    plt.figure(figsize=(10, 6))
    sns.barplot(x=missing_values_to_plot.index, y=missing_values_to_plot.values, palette='viridis')
    plt.title('Number of Missing Values Per Column')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print("No missing values found in the dataset.")
```

```
Missing values in each column:
```

	0
orbital_period	43
mass	522
distance	227

```
dtype: int64
/tmp/ipykernel_4272/3461879747.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `lege

  sns.barplot(x=missing_values_to_plot.index, y=missing_values_to_plot.values, palette='viridis')
```



Task 4: Perform following task on given dataset (Train the model)

- Load the MNIST dataset from PyTorch.
- Display basic information and visualize sample input images with their class labels.
- Perform data preprocessing
- Design a EfficientNet model for image classification.
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics.

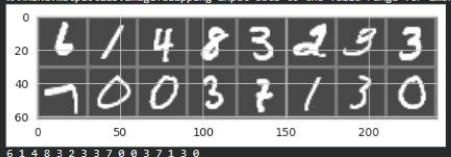
• Visualize the training history (accuracy and loss curves).

```python
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Define transformations for the dataset
transform = transforms.Compose([
    transforms.ToTensor(), # Convert PIL Image or numpy.ndarray to tensor
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize with MNIST dataset's mean and std
])

# Load MNIST training and test datasets
train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

# Create data loaders
batch_size = 64
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f"Number of training samples: {len(train_dataset)}")
print(f"Number of test samples: {len(test_dataset)}")
print(f"Image shape: {train_dataset[0][0].shape}")
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 58.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.68MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 13.8MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 8.31MB/s]
Number of training samples: 60000
Number of test samples: 10000
Image shape: torch.Size([1, 28, 28])
```

```python
def imshow(img):
    img = img / 2 + 0.5  # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# Get some random training images
dataiter = iter(train_loader)
images, labels = next(dataiter)

# Create a grid of images
img_grid = torchvision.utils.make_grid(images[:16]) # Display first 16 images in the batch

# Show images
imshow(img_grid)

# Print labels
print(' '.join(f'{labels[j].item()}' for j in range(16)))
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.28789353..1.9107434].
```




```
6 1 4 8 3 2 3 3 7 0 0 3 7 1 3 0
```

Data preprocessing for the MNIST dataset primarily involves converting images to tensors and normalizing their pixel values. This was handled by the `transforms.Compose` function when loading the dataset. Normalization helps in optimizing model training.

```python
import torch.optim as optim
import torch.nn.functional as F
import torch.nn as nn # Added missing import
# Define Loss function and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# Lists to store training history
train_losses = []
train_accuracies = []
# Train the model for 10 epochs
num_epochs = 10
print(f"Training the model for {num_epochs} epochs...")
for epoch in range(num_epochs):
    model.train() # Set the model to training mode
    running_loss = 0.0
    correct_predictions = 0
    total_samples = 0
    for i, data in enumerate(train_loader, 0):
        inputs, labels = data[0].to(device), data[1].to(device)
        optimizer.zero_grad() # Zero the parameter gradients
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total_samples += labels.size(0)
        correct_predictions += (predicted == labels).sum().item()
    epoch_loss = running_loss / len(train_loader)
    epoch_accuracy = 100 * correct_predictions / total_samples
    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_accuracy)
    print(f'Epoch {epoch+1}/{num_epochs}, Loss: {epoch_loss:.4f}, Accuracy: {epoch_accuracy:.2f}%')
print('Finished Training')
```

```
Training the model for 10 epochs...
Epoch 1/10, Loss: 1.2521, Accuracy: 58.81%
Epoch 2/10, Loss: 1.1495, Accuracy: 62.43%
Epoch 3/10, Loss: 1.1477, Accuracy: 62.58%
Epoch 4/10, Loss: 1.1508, Accuracy: 62.57%
Epoch 5/10, Loss: 1.1498, Accuracy: 62.21%
Epoch 6/10, Loss: 1.1451, Accuracy: 62.43%
Epoch 7/10, Loss: 1.1547, Accuracy: 62.27%
Epoch 8/10, Loss: 1.1536, Accuracy: 62.40%
Epoch 9/10, Loss: 1.1553, Accuracy: 62.23%
Epoch 10/10, Loss: 1.1496, Accuracy: 62.42%
Finished Training
```

```python
import torch.nn as nn
import torchvision.models as models

# Design an EfficientNet model
class EfficientNetClassifier(nn.Module):
    def __init__(self, num_classes=10):
        super(EfficientNetClassifier, self).__init__()
        # Load a pre-trained EfficientNetB0 model
        self.efficientnet = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)

        # Freeze all parameters in the feature extractor
        for param in self.efficientnet.parameters():
            param.requires_grad = False

        # Replace the classifier head for MNIST (10 classes)
        # EfficientNet's classifier is at self.efficientnet.classifier
        # It's a Sequential layer, where the last layer is a Linear layer.
        # The input features for the new classifier need to match the output features of the last pooling layer.

        # The last layer of the original classifier is a Linear layer
        num_ftrs = self.efficientnet.classifier[1].in_features
        self.efficientnet.classifier = nn.Sequential(
            nn.Dropout(p=0.2, inplace=True),
            nn.Linear(num_ftrs, num_classes)
        )

    def forward(self, x):
        # EfficientNet expects 3-channel input, so we replicate the single channel
        if x.shape[1] == 1: # If input is 1 channel (like MNIST)
            x = x.repeat(1, 3, 1, 1)
        return self.efficientnet(x)

# Instantiate the model
model = EfficientNetClassifier(num_classes=10)

# Print the model architecture to verify changes
print(model)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Model moved to: {device}")
```

```
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
              (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            )
```

Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

• Handle missing values in columns (age, income, city).

• Remove duplicate records.

• Standardize text values (e.g., city names in lowercase).

• Encode categorical variables (gender, city).

Sample Code:

import pandas as pd

data = pd.read_csv("shopping_trends.csv")

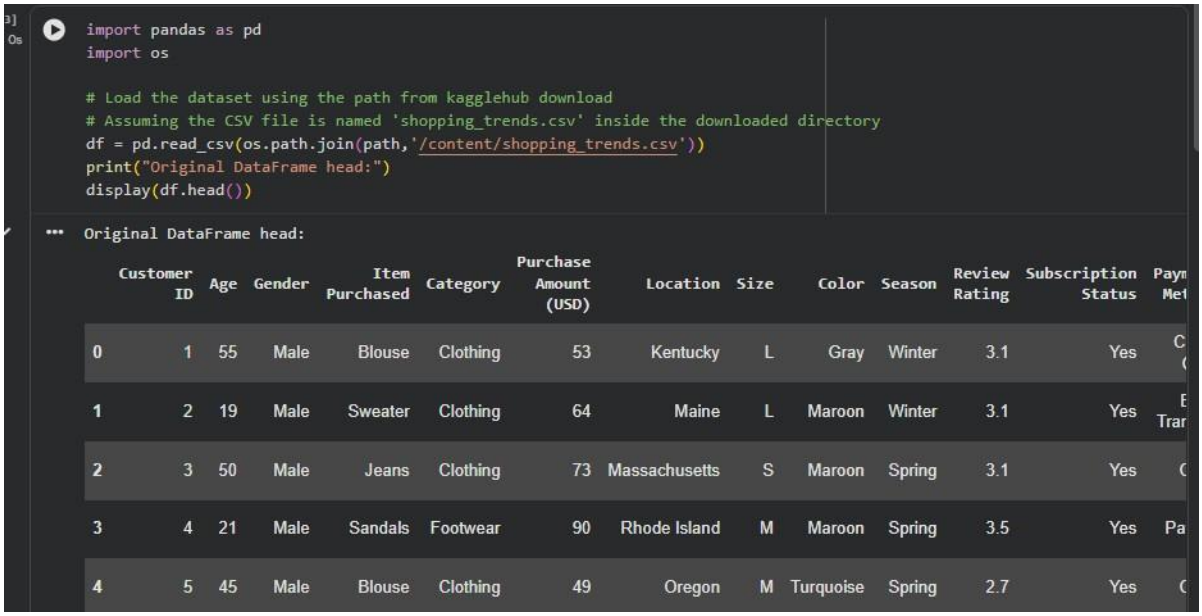
print(data.head())

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-ataset?select=shopping_trends.csv

Expected Output:

• A cleaned Pandas data frame ready for analysis.



```
import pandas as pd
import os

# Load the dataset using the path from kagglehub download
# Assuming the CSV file is named 'shopping_trends.csv' inside the downloaded directory
df = pd.read_csv(os.path.join(path,'/content/shopping_trends.csv'))
print("Original DataFrame head:")
display(df.head())
```

```
Original DataFrame head:
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payn Met
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	C (
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	E Trar
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	C
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Pa
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	C

```python
# Check for missing values before cleaning
print("Missing values before cleaning:")
display(df[['Age', 'Location']].isnull().sum())

# Handle missing values in 'Age' with the median
median_age = df['Age'].median()
df['Age'].fillna(median_age, inplace=True)

# Handle missing values in 'Location' with the mode
mode_city = df['Location'].mode()[0]
df['Location'].fillna(mode_city, inplace=True)

# Re-check for missing values
print("\nMissing values after cleaning (Age, Location):")
display(df[['Age', 'Location']].isnull().sum())
```

```
Missing values before cleaning:
```

	0
Age	0
Location	0

dtype: int64

2. Removing Duplicate Records

We'll identify and remove any duplicate rows from the dataset.

```python
print(f"Number of rows before removing duplicates: {len(df)}")
df.drop_duplicates(inplace=True)
print(f"Number of rows after removing duplicates: {len(df)}")
```

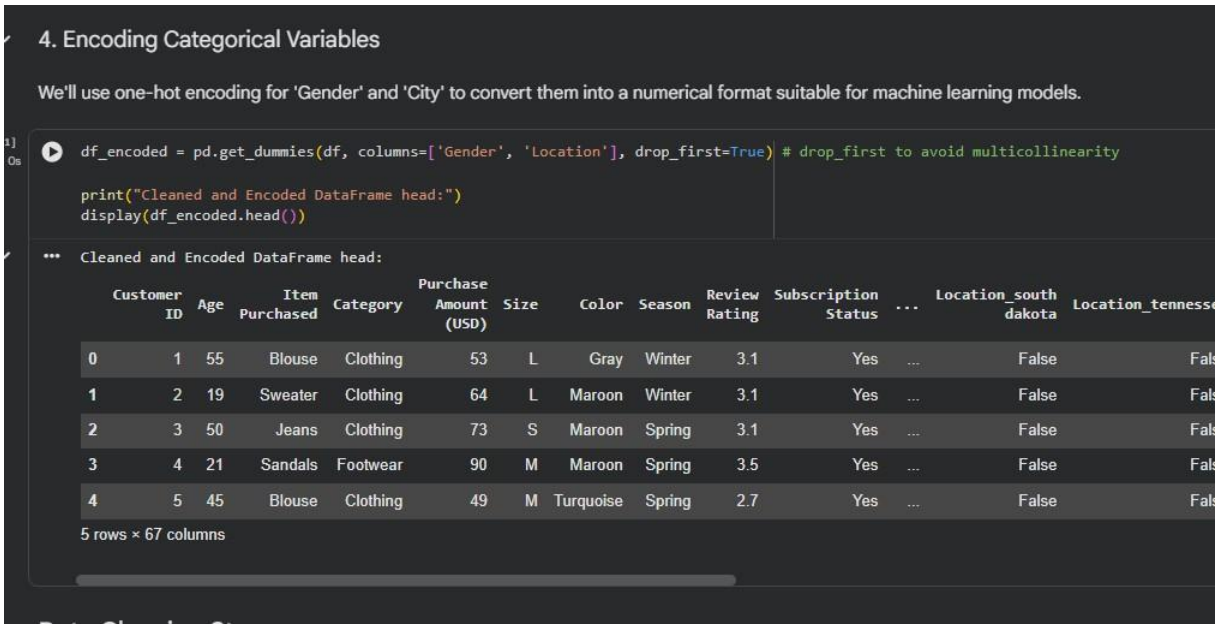
```
Number of rows before removing duplicates: 3900
Number of rows after removing duplicates: 3900
```

3. Standardizing Text Values

We'll convert the 'City' column to lowercase to ensure consistency.

```python
print("Original unique locations (first 5):\n", df['Location'].unique()[:5])
df['Location'] = df['Location'].str.lower()
print("\nStandardized unique locations (first 5):\n", df['Location'].unique()[:5])
```

```
Original unique locations (first 5):
 ['kentucky' 'maine' 'massachusetts' 'rhode island' 'oregon']

Standardized unique locations (first 5):
 ['kentucky' 'maine' 'massachusetts' 'rhode island' 'oregon']
```



4. Encoding Categorical Variables

We'll use one-hot encoding for 'Gender' and 'City' to convert them into a numerical format suitable for machine learning models.

```
df_encoded = pd.get_dummies(df, columns=['Gender', 'Location'], drop_first=True) # drop_first to avoid multicollinearity

print("Cleaned and Encoded DataFrame head:")
display(df_encoded.head())
```

```
Cleaned and Encoded DataFrame head:
```

	Customer ID	Age	Item Purchased	Category	Purchase Amount (USD)	Size	Color	Season	Review Rating	Subscription Status	...	Location_south dakota	Location_tennesse
0	1	55	Blouse	Clothing	53	L	Gray	Winter	3.1	Yes	...	False	Fals
1	2	19	Sweater	Clothing	64	L	Maroon	Winter	3.1	Yes	...	False	Fals
2	3	50	Jeans	Clothing	73	S	Maroon	Spring	3.1	Yes	...	False	Fals
3	4	21	Sandals	Footwear	90	M	Maroon	Spring	3.5	Yes	...	False	Fals
4	5	45	Blouse	Clothing	49	M	Turquoise	Spring	2.7	Yes	...	False	Fals

5 rows × 67 columns

Task 6 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

• Convert date columns into datetime format.

• Create new features (month, year, day of week).

• Normalize sales amount column.

• Detect and handle outliers in dataset.

Sample Code:

```
import pandas as pd

data = pd.read_csv("sales_data.csv")

print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

• A transformed dataset with time-based features and normalized values.

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load the sales data
df = pd.read_csv('/content/sales_data.csv')
print("Original DataFrame head:")
display(df.head())
```

```
Original DataFrame head:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

```python
# Assuming the date column is named 'OrderDate' or similar. Adjust if needed.
# If there are multiple date-like columns, you might need to specify which one.
# For this example, let's assume 'OrderDate' is the primary date column.

if 'Sale_Date' in df.columns:
    df['Sale_Date'] = pd.to_datetime(df['Sale_Date'], errors='coerce')
    # Drop rows where Sale_Date couldn't be parsed
    df.dropna(subset=['Sale_Date'], inplace=True)

    # Create new features
    df['year'] = df['Sale_Date'].dt.year
    df['month'] = df['Sale_Date'].dt.month
    df['day_of_week'] = df['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6
    print("DataFrame with new date features:")
    display(df.head())
else:
    print("Could not find a 'Sale_Date' column. Please check your CSV and adjust the column name if necessary.")
```

```
DataFrame with new date features:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	year	month	day
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2023	2	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	2023	4	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	2023	9	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	2023	8	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	2023	3	

```python
# Assuming the sales amount column is named 'SaleAmount' or 'total_sale_dollars'. Adjust if needed.
# For this example, let's assume 'SaleAmount'.

if 'Sales_Amount' in df.columns:
    # Initialize MinMaxScaler
    scaler = MinMaxScaler()
    df['NormalizedSaleAmount'] = scaler.fit_transform(df[['Sales_Amount']])
    print("DataFrame with normalized sales amount:")
    display(df.head())
else:
    print("Could not find a 'Sales_Amount' column. Please check your CSV and adjust the column name if necessary.")
```

```
DataFrame with normalized sales amount:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	year	month	d
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2023	2	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	2023	4	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	2023	9	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	2023	8	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	2023	3	

```python
if 'Sales_Amount' in df.columns:
    # Calculate IQR for 'Sales_Amount'
    Q1 = df['Sales_Amount'].quantile(0.25)
    Q3 = df['Sales_Amount'].quantile(0.75)
    IQR = Q3 - Q1

    # Define outlier bounds
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Detect outliers
    outliers = df[(df['Sales_Amount'] < lower_bound) | (df['Sales_Amount'] > upper_bound)]
    print(f"Number of outliers detected in 'Sales_Amount': {len(outliers)}")
    if not outliers.empty:
        print("Sample of outliers:")
        display(outliers.head())

    # Handle outliers by capping them to the bounds
    df['Sales_Amount_Capped'] = df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)
    print("DataFrame with 'Sales_Amount' outliers capped:")
    display(df.head())
else:
    print("Skipping outlier detection as 'Sales_Amount' column was not found.")
```

```
Number of outliers detected in 'Sales_Amount': 0
DataFrame with 'Sales_Amount' outliers capped:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	year	month	day
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2023	2	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	2023	4	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	2023	9	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	2023	8	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	2023	3	

```
print("Final transformed DataFrame head with new features, normalized sales, and capped outliers:")
display(df.head())
print("DataFrame Info after transformations:")
df.info()
```

```
Final transformed DataFrame head with new features, normalized sales, and capped outliers:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	year	month	da
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2023	2	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	2023	4	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	2023	9	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	2023	8	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	2023	3	

```
DataFrame Info after transformations:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Product_ID           1000 non-null   int64
 1   Sale_Date            1000 non-null   datetime64[ns]
 2   Sales_Rep            1000 non-null   object
 3   Region               1000 non-null   object
 4   Sales_Amount         1000 non-null   float64
 5   Quantity_Sold        1000 non-null   int64
 6   Product_Category     1000 non-null   object
 7   Unit_Cost            1000 non-null   float64
 8   Unit_Price           1000 non-null   float64
 9   Customer_Type        1000 non-null   object
 10  Discount             1000 non-null   float64
 11  Payment_Method       1000 non-null   object
 12  Sales_Channel        1000 non-null   object
 13  Region_and_Sales_Rep 1000 non-null   object
 14  year                 1000 non-null   int32
```

Task 7 – Healthcare Data Preparation

Task: Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

• Handle missing values in blood_pressure, cholesterol.

• Encode categorical features

• Scale numerical features (min-max or standard scaling).

• Split dataset into training and testing sets.

Dataset link :https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset

Expected Output: • A preprocessed dataset ready for machine learning models.

```
# Check and handle missing values for 'blood_pressure' and 'cholesterol'
# (assuming these columns might be added later or are conceptually part of the dataset)

missing_cols_to_check = ['blood_pressure', 'cholesterol']

for col in missing_cols_to_check:
    if col in df.columns:
        if df[col].isnull().sum() > 0:
            print(f"Handling missing values in '{col}'...")
            # For numerical columns, impute with the median
            df[col].fillna(df[col].median(), inplace=True)
            print(f"Missing values in '{col}' after imputation: {df[col].isnull().sum()}")
        else:
            print(f"No missing values found in '{col}'.")
    else:
        print(f"Column '{col}' not found in the DataFrame. Skipping missing value handling for this column.")

print("\nDataFrame info after handling missing values:")
df.info()
```

```
Column 'blood_pressure' not found in the DataFrame. Skipping missing value handling for this column.
Column 'cholesterol' not found in the DataFrame. Skipping missing value handling for this column.

DataFrame info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Product_ID           1000 non-null   int64
 1   Sale_Date            1000 non-null   datetime64[ns]
 2   Sales_Rep            1000 non-null   object
 3   Region               1000 non-null   object
 4   Sales_Amount         1000 non-null   float64
 5   Quantity_Sold        1000 non-null   int64
 6   Product_Category     1000 non-null   object
 7   Unit_Cost            1000 non-null   float64
 8   Unit_Price           1000 non-null   float64
 9   Customer_Type        1000 non-null   object
 10  Discount             1000 non-null   float64
 11  Payment_Method       1000 non-null   object
 12  Sales_Channel        1000 non-null   object
 13  Region_and_Sales_Rep 1000 non-null   object
 14  year                 1000 non-null   int32
 15  month                1000 non-null   int32
 16  day_of_week          1000 non-null   int32
```

```python
# Identify categorical columns (object dtype)
categorical_cols = df.select_dtypes(include='object').columns

if not categorical_cols.empty:
    print(f"Categorical columns to encode: {list(categorical_cols)}")
    # Apply one-hot encoding using pd.get_dummies
    df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
    print("\nDataFrame head after one-hot encoding:")
    display(df.head())
else:
    print("No categorical columns found for encoding.")

print("\nDataFrame info after encoding categorical features:")
df.info()
```

```
Categorical columns to encode: ['Sales_Rep', 'Region', 'Product_Category', 'Customer_Type', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']

DataFrame head after one-hot encoding:
```

	Product_ID	Sale_Date	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price	Discount	year	month	day_of_week	...	Region_and_Sales_Rep_South-Alice	Region_and_Sales_Rep_South-Bob	Region_and_Sales_Rep_South-Charlie
0	1052	2023-02-03	5053.97	18	152.75	267.22	0.09	2023	2	4	...	False	False	False
1	1093	2023-04-21	4384.02	17	3816.39	4209.44	0.11	2023	4	4	...	False	False	False
2	1015	2023-09-21	4631.23	30	261.56	371.40	0.20	2023	9	3	...	False	False	False
3	1072	2023-08-24	2167.94	39	4330.03	4467.75	0.02	2023	8	3	...	False	True	False
4	1061	2023-03-24	3750.20	13	637.37	692.71	0.08	2023	3	4	...	False	False	False

5 rows × 45 columns

```
Data columns (total 45 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Product_ID                              1000 non-null   int64
 1   Sale_Date                               1000 non-null   datetime64[ns]
 2   Sales_Amount                            1000 non-null   float64
 3   Quantity_Sold                           1000 non-null   int64
 4   Unit_Cost                               1000 non-null   float64
 5   Unit_Price                              1000 non-null   float64
 6   Discount                                1000 non-null   float64
 7   year                                    1000 non-null   int32
 8   month                                   1000 non-null   int32
 9   day_of_week                             1000 non-null   int32
 10  NormalizedSaleAmount                    1000 non-null   float64
 11  Sales_Amount_Capped                     1000 non-null   float64
 12  Sales_Rep_Bob                           1000 non-null   bool
 13  Sales_Rep_Charlie                       1000 non-null   bool
 14  Sales_Rep_David                         1000 non-null   bool
 15  Sales_Rep_Eve                           1000 non-null   bool
 16  Region_North                            1000 non-null   bool
 17  Region_South                            1000 non-null   bool
 18  Region_West                             1000 non-null   bool
 19  Product_Category_Electronics            1000 non-null   bool
 20  Product_Category_Food                   1000 non-null   bool
 21  Product_Category_Furniture              1000 non-null   bool
 22  Customer_Type_Returning                 1000 non-null   bool
 23  Payment_Method_Cash                     1000 non-null   bool
 24  Payment_Method_Credit Card              1000 non-null   bool
 25  Sales_Channel_Retail                    1000 non-null   bool
 26  Region_and_Sales_Rep_East-Bob           1000 non-null   bool
 27  Region_and_Sales_Rep_East-Charlie       1000 non-null   bool
 28  Region_and_Sales_Rep_East-David         1000 non-null   bool
 29  Region_and_Sales_Rep_East-Eve           1000 non-null   bool
 30  Region_and_Sales_Rep_North-Alice        1000 non-null   bool
 31  Region_and_Sales_Rep_North-Bob          1000 non-null   bool
 32  Region_and_Sales_Rep_North-Charlie      1000 non-null   bool
 33  Region_and_Sales_Rep_North-David        1000 non-null   bool
 34  Region_and_Sales_Rep_North-Eve          1000 non-null   bool
 35  Region_and_Sales_Rep_South-Alice        1000 non-null   bool
 36  Region_and_Sales_Rep_South-Bob          1000 non-null   bool
 37  Region_and_Sales_Rep_South-Charlie      1000 non-null   bool
 38  Region_and_Sales_Rep_South-David        1000 non-null   bool
 39  Region_and_Sales_Rep_South-Eve          1000 non-null   bool
```

```
numerical_cols = df.select_dtypes(include=['int64', 'int32', 'float64']).columns.tolist()

# Remove columns that shouldn't be scaled (e.g., identifiers, already scaled, or target)
if 'Product_ID' in numerical_cols: numerical_cols.remove('Product_ID')
if 'Sales_Amount' in numerical_cols: numerical_cols.remove('Sales_Amount') # Original sales amount
if 'NormalizedSaleAmount' in numerical_cols: numerical_cols.remove('NormalizedSaleAmount') # Already normalized

# If 'Sales_Amount_Capped' is the target, remove it from features to be scaled
# For now, let's assume it might be a feature to be scaled if not explicitly designated as target yet
# If it is the target, it will be handled when splitting X and y.

if numerical_cols:
    print(f"Numerical columns to scale: {numerical_cols}")
    scaler = MinMaxScaler()
    df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
    print("\nDataFrame head after numerical feature scaling:")
    display(df.head())
else:
    print("No numerical columns found for scaling (besides those already handled or excluded).")

print("\nDataFrame info after scaling numerical features:")
df.info()
```

```
Numerical columns to scale: ['Quantity_Sold', 'Unit_Cost', 'Unit_Price', 'Discount', 'year', 'month', 'day_of_week', 'Sales_Amount_Capped']

DataFrame head after numerical feature scaling:
```

	Product_ID	Sale_Date	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price	Discount	year	month	day_of_week	...	Region_and_Sales_Rep_South-Alice	Region_and_Sales_Rep_South-Bob	Region_and_Sales_Rep_South-Charlie
0	1052	2023-02-03	5053.97	0.354167	0.018738	0.018976	0.300000	0.0	0.090909	0.666667	...	False	False	False
1	1093	2023-04-21	4384.02	0.333333	0.761113	0.766312	0.366667	0.0	0.272727	0.666667	...	False	False	False
2	1015	2023-09-21	4631.23	0.604167	0.040786	0.038726	0.666667	0.0	0.727273	0.500000	...	False	False	False
3	1072	2023-08-24	2167.94	0.791667	0.865194	0.815281	0.066667	0.0	0.636364	0.500000	...	False	True	False
4	1061	2023-03-24	3750.20	0.250000	0.116938	0.099637	0.266667	0.0	0.181818	0.666667	...	False	False	False

5 rows × 45 columns

```
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
# Assuming 'Sales_Amount_Capped' is the target variable for a regression task
# Drop original 'Sales_Amount' and 'Sale_Date' (if still present and not needed as features)

y = df['Sales_Amount_Capped']

X = df.drop(columns=['Sales_Amount_Capped', 'Sales_Amount', 'Sale_Date', 'Product_ID'], errors='ignore')

print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"\nX_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nDataset split into training and testing sets successfully!")
```

```
Features (X) shape: (1000, 41)
Target (y) shape: (1000,)

X_train shape: (800, 41)
X_test shape: (200, 41)
y_train shape: (800,)
y_test shape: (200,)

Dataset split into training and testing sets successfully!
```

Task 8 – Employee Salary Data Preprocessing

Task: Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

• Handle missing values

• Detect and remove salary outliers.

• Standardize department names (lowercase).

• Apply label encoding to job location, department.

Dataset link: https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output: A structured dataset ready for HR analytics.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load the employee salary data for HR analytics
hr_df = pd.read_csv('/content/employee_salary_dataset.csv')
print("Original HR DataFrame head:")
display(hr_df.head())
print("\nOriginal HR DataFrame info:")
hr_df.info()
```

```
Original HR DataFrame head:
```

	EmployeeID	Name	Department	Experience_Years	Education_Level	Age	Gender	City	Monthly_Salary
0	1	Employee_1	Marketing	15	Master	53	Female	Delhi	111416
1	2	Employee_2	Operations	7	Bachelor	25	Female	Bangalore	95271
2	3	Employee_3	IT	12	High School	51	Female	Hyderabad	69064
3	4	Employee_4	Operations	8	PhD	44	Male	Delhi	95091
4	5	Employee_5	Operations	15	Master	36	Female	Delhi	132450

```
Original HR DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   EmployeeID       50 non-null     int64
 1   Name             50 non-null     object
 2   Department       50 non-null     object
 3   Experience_Years 50 non-null     int64
 4   Education_Level  50 non-null     object
 5   Age              50 non-null     int64
 6   Gender           50 non-null     object
 7   City             50 non-null     object
 8   Monthly_Salary   50 non-null     int64
dtypes: int64(4), object(5)
memory usage: 3.6+ KB
```

1. Handle missing values

```
print("Missing values before handling:")
display(hr_df.isnull().sum())

# Iterate through columns and handle missing values
for col in hr_df.columns:
    if hr_df[col].isnull().any():
        if hr_df[col].dtype in ['int64', 'float64']:
            # Impute numerical columns with the median
            hr_df[col].fillna(hr_df[col].median(), inplace=True)
            print(f"Imputed numerical column '{col}' with median.")
        else:
            # Impute categorical columns with the mode
            mode_val = hr_df[col].mode()[0]
            hr_df[col].fillna(mode_val, inplace=True)
            print(f"Imputed categorical column '{col}' with mode: {mode_val}.")

print("\nMissing values after handling:")
display(hr_df.isnull().sum())
print("\nHR DataFrame info after handling missing values:")
hr_df.info()
```

```
Missing values before handling:
```

	0
EmployeeID	0
Name	0
Department	0
Experience_Years	0
Education_Level	0
Age	0
Gender	0
City	0
Monthly_Salary	0

```
dtype: int64

Missing values after handling:
```

```
Missing values after handling:
```

	0
EmployeeID	0
Name	0
Department	0
Experience_Years	0
Education_Level	0
Age	0
Gender	0
City	0
Monthly_Salary	0

```
dtype: int64

HR DataFrame info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   EmployeeID        50 non-null     int64
 1   Name              50 non-null     object
 2   Department        50 non-null     object
 3   Experience_Years  50 non-null     int64
 4   Education_Level   50 non-null     object
 5   Age               50 non-null     int64
 6   Gender            50 non-null     object
 7   City              50 non-null     object
 8   Monthly_Salary    50 non-null     int64
dtypes: int64(4), object(5)
memory usage: 3.6+ KB
```

```python
if 'Monthly_Salary' in hr_df.columns:
    # Calculate IQR for 'Monthly_Salary'
    Q1_salary = hr_df['Monthly_Salary'].quantile(0.25)
    Q3_salary = hr_df['Monthly_Salary'].quantile(0.75)
    IQR_salary = Q3_salary - Q1_salary

    # Define outlier bounds
    lower_bound_salary = Q1_salary - 1.5 * IQR_salary
    upper_bound_salary = Q3_salary + 1.5 * IQR_salary

    # Detect outliers
    salary_outliers = hr_df[(hr_df['Monthly_Salary'] < lower_bound_salary) | (hr_df['Monthly_Salary'] > upper_bound_salary)]
    print(f"Number of salary outliers detected: {len(salary_outliers)}")
    if not salary_outliers.empty:
        print("Sample of salary outliers before removal:")
        display(salary_outliers.head())

    # Remove outliers
    hr_df = hr_df[~((hr_df['Monthly_Salary'] < lower_bound_salary) | (hr_df['Monthly_Salary'] > upper_bound_salary))]
    print(f"\nHR DataFrame shape after removing salary outliers: {hr_df.shape}")
    print("HR DataFrame head after removing salary outliers:")
    display(hr_df.head())
else:
    print("Could not find a 'Monthly_Salary' column for outlier detection. Skipping this step.")
```

```
Number of salary outliers detected: 0

HR DataFrame shape after removing salary outliers: (50, 10)
HR DataFrame head after removing salary outliers:
```

	EmployeeID	Name	Department	Experience_Years	Education_Level	Age	Gender	City	Monthly_Salary	Department_Encoded
0	1	Employee_1	marketing	15	Master	53	Female	Delhi	111416	3
1	2	Employee_2	operations	7	Bachelor	25	Female	Bangalore	95271	4
2	3	Employee_3	it	12	High School	51	Female	Hyderabad	69064	2
3	4	Employee_4	operations	8	PhD	44	Male	Delhi	95091	4
4	5	Employee_5	operations	15	Master	36	Female	Delhi	132450	4

```python
if 'Department' in hr_df.columns:
    hr_df['Department'] = hr_df['Department'].str.lower()
    print("Department names standardized to lowercase:")
    display(hr_df['Department'].value_counts())
else:
    print("Could not find 'Department' column. Skipping standardization.")
```

```
Department names standardized to lowercase:
```

Department	count
marketing	13
operations	10
it	10
finance	10
hr	7

dtype: int64

```python
le = LabelEncoder()
columns_to_encode = ['City', 'Department']
for col in columns_to_encode:
    if col in hr_df.columns:
        hr_df[f'{col}_Encoded'] = le.fit_transform(hr_df[col])
        print(f"'{col}' encoded. Original values vs. Encoded values:")
        display(hr_df[[col, f'{col}_Encoded']].head())
    else:
        print(f"Column '{col}' not found for encoding. Skipping.")
print("\nHR DataFrame head after label encoding:")
display(hr_df.head())
print("\nHR DataFrame info after label encoding:")
hr_df.info()
```

```
'City' encoded. Original values vs. Encoded values:
```

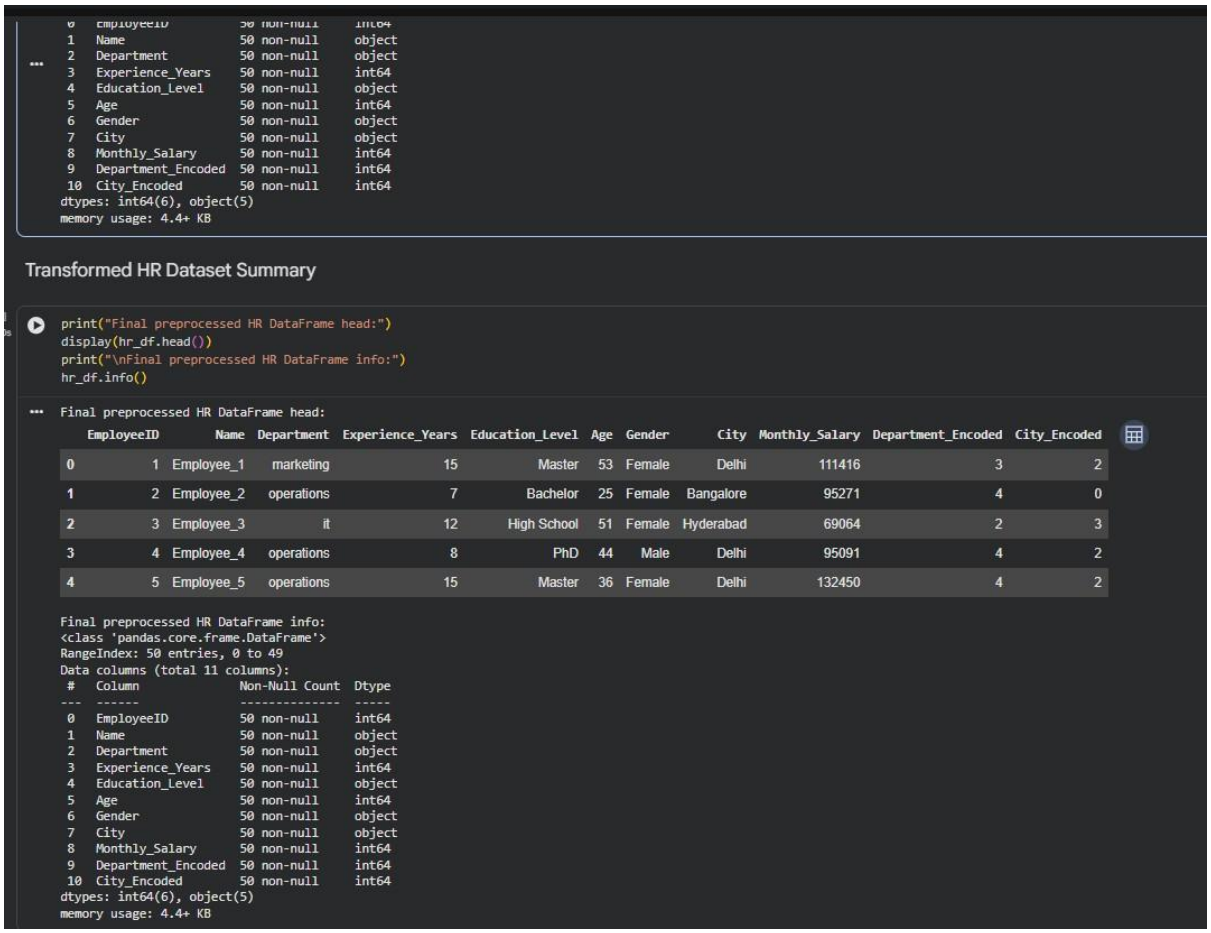
	City	City_Encoded
0	Delhi	2
1	Bangalore	0
2	Hyderabad	3
3	Delhi	2
4	Delhi	2

```
'Department' encoded. Original values vs. Encoded values:
```

	Department	Department_Encoded
0	marketing	3
1	operations	4
2	it	2
3	operations	4
4	operations	4

```
HR DataFrame head after label encoding:
```

	EmployeeID	Name	Department	Experience_Years	Education_Level	Age	Gender	City	Monthly_Salary	Department_Encoded	City_Encoded
0	1	Employee_1	marketing	15	Master	53	Female	Delhi	111416	3	2
1	2	Employee_2	operations	7	Bachelor	25	Female	Bangalore	95271	4	0
2	3	Employee_3	it	12	High School	51	Female	Hyderabad	69064	2	3
3	4	Employee_4	operations	8	PhD	44	Male	Delhi	95091	4	2
4	5	Employee_5	operations	15	Master	36	Female	Delhi	132450	4	2



```
 0   EmployeeID          50 non-null     int64
 1   Name                50 non-null     object
 2   Department          50 non-null     object
 3   Experience_Years    50 non-null     int64
 4   Education_Level     50 non-null     object
 5   Age                 50 non-null     int64
 6   Gender              50 non-null     object
 7   City                50 non-null     object
 8   Monthly_Salary      50 non-null     int64
 9   Department_Encoded  50 non-null     int64
 10  City_Encoded        50 non-null     int64
dtypes: int64(6), object(5)
memory usage: 4.4+ KB
```

Transformed HR Dataset Summary

```
print("Final preprocessed HR DataFrame head:")
display(hr_df.head())
print("\nFinal preprocessed HR DataFrame info:")
hr_df.info()
```

Final preprocessed HR DataFrame head:

	EmployeeID	Name	Department	Experience_Years	Education_Level	Age	Gender	City	Monthly_Salary	Department_Encoded	City_Encoded
0	1	Employee_1	marketing	15	Master	53	Female	Delhi	111416	3	2
1	2	Employee_2	operations	7	Bachelor	25	Female	Bangalore	95271	4	0
2	3	Employee_3	it	12	High School	51	Female	Hyderabad	69064	2	3
3	4	Employee_4	operations	8	PhD	44	Male	Delhi	95091	4	2
4	5	Employee_5	operations	15	Master	36	Female	Delhi	132450	4	2

```
Final preprocessed HR DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   EmployeeID          50 non-null     int64
 1   Name                50 non-null     object
 2   Department          50 non-null     object
 3   Experience_Years    50 non-null     int64
 4   Education_Level     50 non-null     object
 5   Age                 50 non-null     int64
 6   Gender              50 non-null     object
 7   City                50 non-null     object
 8   Monthly_Salary      50 non-null     int64
 9   Department_Encoded  50 non-null     int64
 10  City_Encoded        50 non-null     int64
dtypes: int64(6), object(5)
memory usage: 4.4+ KB
```

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

• Handle missing values in loan_amount, credit_score.

• Encode loan_status (Approved/Rejected).

• Scale numerical features using standard scaling.

• Split the dataset into training and testing sets.

Sample Code:

import pandas as pd

data = pd.read_csv("loadapproval.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/amineipad/loan-approval-dataset

Expected Output:

• A machine-learning-ready loan dataset.

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# Load the loan approval data
loan_df = pd.read_csv('/content/loanapproval.csv')
print("Original Loan DataFrame head:")
display(loan_df.head())
print("\nOriginal Loan DataFrame info:")
loan_df.info()
```

```
Original Loan DataFrame head:
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approved
0	1	59	Male	Divorced	100073	7169	793	1	1	Unemployed	1
1	2	49	Male	Married	112197	23556	789	0	2	Employed	1
2	3	35	Male	Divorced	84429	27052	372	1	4	Unemployed	0
3	4	63	Female	Single	124195	11313	808	3	4	Self-employed	1
4	5	28	Female	Married	81627	13315	689	0	1	Unemployed	1

```
Original Loan DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   applicant_id         1000 non-null   int64
 1   age                  1000 non-null   int64
 2   gender               1000 non-null   object
 3   marital_status       1000 non-null   object
 4   annual_income        1000 non-null   int64
 5   loan_amount          1000 non-null   int64
 6   credit_score         1000 non-null   int64
 7   num_dependents       1000 non-null   int64
 8   existing_loans_count 1000 non-null   int64
 9   employment_status    1000 non-null   object
 10  loan_approved        1000 non-null   int64
dtypes: int64(8), object(3)
memory usage: 86.1+ KB
```

```python
missing_cols_loan = ['loan_amount', 'credit_score']

print("Missing values before handling:")
display(loan_df[missing_cols_loan].isnull().sum())

for col in missing_cols_loan:
    if col in loan_df.columns:
        if loan_df[col].isnull().sum() > 0:
            print(f"Handling missing values in '{col}'...")
            # For numerical columns, impute with the median
            loan_df[col].fillna(loan_df[col].median(), inplace=True)
            print(f"Missing values in '{col}' after imputation: {loan_df[col].isnull().sum()}")
        else:
            print(f"No missing values found in '{col}'.")
    else:
        print(f"Column '{col}' not found in the DataFrame. Skipping missing value handling for this column.")

print("\nLoan DataFrame info after handling missing values:")
loan_df.info()
```

```
Missing values before handling:
```

	0
loan_amount	0
credit_score	0

```
dtype: int64
No missing values found in 'loan_amount'.
No missing values found in 'credit_score'.

Loan DataFrame info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   applicant_id         1000 non-null   int64
 1   age                  1000 non-null   int64
 2   gender               1000 non-null   object
 3   marital_status       1000 non-null   object
 4   annual_income        1000 non-null   int64
 5   loan_amount          1000 non-null   int64
 6   credit_score         1000 non-null   int64
 7   num_dependents       1000 non-null   int64
 8   existing_loans_count 1000 non-null   int64
 9   employment_status    1000 non-null   object
 10  loan_approved        1000 non-null   int64
dtypes: int64(8), object(3)
memory usage: 86.1+ KB
```

```python
if 'loan_status' in loan_df.columns:
    le_loan_status = LabelEncoder()
    loan_df['loan_status_Encoded'] = le_loan_status.fit_transform(loan_df['loan_status'])
    print("'loan_status' encoded. Original values vs. Encoded values:")
    display(loan_df[['loan_status', 'loan_status_Encoded']].head())
else:
    print("Column 'loan_status' not found for encoding. Skipping.")

print("\nLoan DataFrame head after encoding 'loan_status':")
display(loan_df.head())
loan_df.info()
```

```
Column 'loan_status' not found for encoding. Skipping.

Loan DataFrame head after encoding 'loan_status':
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approved
0	1	59	Male	Divorced	100073	7169	793	1	1	Unemployed	1
1	2	49	Male	Married	112197	23556	789	0	2	Employed	1
2	3	35	Male	Divorced	84429	27052	372	1	4	Unemployed	0
3	4	63	Female	Single	124195	11313	808	3	4	Self-employed	1
4	5	28	Female	Married	81627	13315	689	0	1	Unemployed	1

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   applicant_id         1000 non-null   int64
 1   age                  1000 non-null   int64
 2   gender               1000 non-null   object
 3   marital_status       1000 non-null   object
 4   annual_income        1000 non-null   int64
 5   loan_amount          1000 non-null   int64
 6   credit_score         1000 non-null   int64
 7   num_dependents       1000 non-null   int64
 8   existing_loans_count 1000 non-null   int64
 9   employment_status    1000 non-null   object
 10  loan_approved        1000 non-null   int64
dtypes: int64(8), object(3)
memory usage: 86.1+ KB
```

```python
scaler_loan = StandardScaler()

# Identify numerical columns for scaling, excluding 'loan_status_Encoded' (target) and any IDs
numerical_cols_loan = loan_df.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Remove columns that should not be scaled (e.g., target or identifiers)
if 'loan_status_Encoded' in numerical_cols_loan: numerical_cols_loan.remove('loan_status_Encoded')
if 'customer_id' in numerical_cols_loan: numerical_cols_loan.remove('customer_id')

if numerical_cols_loan:
    print(f"Numerical columns to scale: {numerical_cols_loan}")
    loan_df[numerical_cols_loan] = scaler_loan.fit_transform(loan_df[numerical_cols_loan])
    print("\nLoan DataFrame head after numerical feature scaling:")
    display(loan_df.head())
else:
    print("No numerical columns found for scaling (besides those excluded or already handled).")

print("\nLoan DataFrame info after scaling numerical features:")
loan_df.info()
```

```
Numerical columns to scale: ['applicant_id', 'age', 'annual_income', 'loan_amount', 'credit_score', 'num_dependents', 'existing_loans_count', 'loan_approved']

Loan DataFrame head after numerical feature scaling:
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approved
0	-1.730320	1.307840	Male	Divorced	0.482301	-1.566427	1.391620	-0.689042	-0.737500	Unemployed	0.609707
1	-1.726856	0.514489	Male	Married	0.805362	-0.287825	1.365954	-1.394305	-0.029726	Employed	0.609707
2	-1.723391	-0.596204	Male	Divorced	0.065444	-0.015048	-1.309808	-0.689042	1.385820	Unemployed	-1.640133
3	-1.719927	1.625181	Female	Single	1.125066	-1.243090	1.487871	0.721484	1.385820	Self-employed	0.609707
4	-1.716463	-1.151550	Female	Married	-0.009219	-1.086883	0.724284	-1.394305	-0.737500	Unemployed	0.609707

```
Loan DataFrame info after scaling numerical features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   applicant_id         1000 non-null   float64
 1   age                  1000 non-null   float64
 2   gender               1000 non-null   object
 3   marital_status       1000 non-null   object
 4   annual_income        1000 non-null   float64
 5   loan_amount          1000 non-null   float64
 6   credit_score         1000 non-null   float64
 7   num_dependents       1000 non-null   float64
 8   existing_loans_count 1000 non-null   float64
 9   employment_status    1000 non-null   object
 10  loan_approved        1000 non-null   float64
dtypes: float64(8), object(3)
memory usage: 86.1+ KB
```

```python
from sklearn.model_selection import train_test_split
import pandas as pd

# Define features (X_loan) and target (y_loan)
# 'loan_approved' is the target variable, which is already 0/1 encoded.
y_loan = loan_df['loan_approved']

# Drop target, original categorical columns (which will be one-hot encoded), and identifiers from features
X_loan = loan_df.drop(columns=['loan_approved', 'applicant_id'], errors='ignore')

# Identify remaining categorical columns in X_loan for one-hot encoding
categorical_cols_x_loan = X_loan.select_dtypes(include='object').columns

if not categorical_cols_x_loan.empty:
    print(f"Categorical columns in X_loan to one-hot encode: {list(categorical_cols_x_loan)}")
    X_loan = pd.get_dummies(X_loan, columns=categorical_cols_x_loan, drop_first=True)
else:
    print("No categorical columns found in X_loan for one-hot encoding.")

print(f"Features (X_loan) shape: {X_loan.shape}")
print(f"Target (y_loan) shape: {y_loan.shape}")

# Split the data into training and testing sets
X_loan_train, X_loan_test, y_loan_train, y_loan_test = train_test_split(X_loan, y_loan, test_size=0.2, random_state=42, stratify=y_loan)

print(f"\nX_loan_train shape: {X_loan_train.shape}")
print(f"X_loan_test shape: {X_loan_test.shape}")
print(f"y_loan_train shape: {y_loan_train.shape}")
print(f"y_loan_test shape: {y_loan_test.shape}")

print("\nLoan dataset split into training and testing sets successfully!")
```

```
Categorical columns in X_loan to one-hot encode: ['gender', 'marital_status', 'employment_status']
Features (X_loan) shape: (1000, 11)
Target (y_loan) shape: (1000,)

X_loan_train shape: (800, 11)
X_loan_test shape: (200, 11)
y_loan_train shape: (800,)
y_loan_test shape: (200,)

Loan dataset split into training and testing sets successfully!
```

```python
print("Final preprocessed Loan DataFrame head:")
display(loan_df.head())
print("\nFinal preprocessed Loan DataFrame info:")
loan_df.info()
```

Final preprocessed Loan DataFrame head:

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approved
0	-1.730320	1.307840	Male	Divorced	0.482301	-1.566427	1.391620	-0.689042	-0.737500	Unemployed	0.609707
1	-1.726856	0.514489	Male	Married	0.805362	-0.287825	1.365954	-1.394305	-0.029726	Employed	0.609707
2	-1.723391	-0.596204	Male	Divorced	0.065444	-0.015048	-1.309808	-0.689042	1.385820	Unemployed	-1.640133
3	-1.719927	1.625181	Female	Single	1.125066	-1.243090	1.487871	0.721484	1.385820	Self-employed	0.609707
4	-1.716463	-1.151550	Female	Married	-0.009219	-1.086883	0.724284	-1.394305	-0.737500	Unemployed	0.609707

```
Final preprocessed Loan DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   applicant_id         1000 non-null   float64
 1   age                  1000 non-null   float64
 2   gender               1000 non-null   object
 3   marital_status       1000 non-null   object
 4   annual_income        1000 non-null   float64
 5   loan_amount          1000 non-null   float64
 6   credit_score         1000 non-null   float64
 7   num_dependents       1000 non-null   float64
 8   existing_loans_count 1000 non-null   float64
 9   employment_status    1000 non-null   object
 10  loan_approved        1000 non-null   float64
dtypes: float64(8), object(3)
memory usage: 86.1+ KB
```

Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- Remove duplicate user profiles.
- Handle missing values
- Encode categorical columns

Dataset link:

https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles

Expected Output:

• A cleaned dataset for engagement prediction.

```
import pandas as pd

# Load the social media engagement data with a specified encoding
social_df = pd.read_csv('/content/SocialMediaTop100.csv', encoding='latin1')
print("Original Social Media DataFrame head:")
display(social_df.head())
print("\nOriginal Social Media DataFrame info:")
social_df.info()
```

```
Original Social Media DataFrame head:
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

```
Original Social Media DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   N          500 non-null    int64
 1   PROFILE    500 non-null    object
 2   FOLLOWERS  500 non-null    float64
 3   POSTS      400 non-null    float64
 4   Platform   500 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 19.7+ KB
```

```
# Assuming 'PROFILE' uniquely identifies a user profile. Adjust if needed.
initial_rows = social_df.shape[0]
social_df.drop_duplicates(subset=['PROFILE'], inplace=True)

print(f"Number of rows before removing duplicates: {initial_rows}")
print(f"Number of rows after removing duplicates: {social_df.shape[0]}")
print("\nSocial Media DataFrame head after removing duplicates:")
display(social_df.head())
```

```
Number of rows before removing duplicates: 500
Number of rows after removing duplicates: 465

Social Media DataFrame head after removing duplicates:
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

```
print("Missing values before handling:")
display(social_df.isnull().sum())

# Iterate through columns and handle missing values
for col in social_df.columns:
    if social_df[col].isnull().any():
        if social_df[col].dtype in ['int64', 'float64']:
            # Impute numerical columns with the median
            social_df[col].fillna(social_df[col].median(), inplace=True)
            print(f"Imputed numerical column '{col}' with median.")
        else:
            # Impute categorical columns with the mode
            mode_val = social_df[col].mode()[0]
            social_df[col].fillna(mode_val, inplace=True)
            print(f"Imputed categorical column '{col}' with mode: {mode_val}.")

print("\nMissing values after handling:")
display(social_df.isnull().sum())
print("\nSocial Media DataFrame info after handling missing values:")
social_df.info()
```

```
Missing values before handling:
```

	0
N	0
PROFILE	0
FOLLOWERS	0
POSTS	99
Platform	0

```
dtype: int64
Imputed numerical column 'POSTS' with median.

Missing values after handling:
/tmp/ipykernel_7361/781838298.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  social_df[col].fillna(social_df[col].median(), inplace=True)
```

	0
N	0
PROFILE	0
FOLLOWERS	0
POSTS	0
Platform	0

```
dtype: int64

Social Media DataFrame info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
Index: 465 entries, 0 to 499
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   N          465 non-null    int64
 1   PROFILE    465 non-null    object
 2   FOLLOWERS  465 non-null    float64
 3   POSTS      465 non-null    float64
 4   Platform   465 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 21.8+ KB
```

```
# Identify categorical columns (object dtype)
categorical_cols_social = social_df.select_dtypes(include='object').columns

if not categorical_cols_social.empty:
    print(f"Categorical columns to encode: {list(categorical_cols_social)}")
    # Apply one-hot encoding using pd.get_dummies
    social_df = pd.get_dummies(social_df, columns=categorical_cols_social, drop_first=True)
    print("\nSocial Media DataFrame head after one-hot encoding:")
    display(social_df.head())
else:
    print("No categorical columns found for encoding.")

print("\nSocial Media DataFrame info after encoding categorical features:")
social_df.info()
```

```
Categorical columns to encode: ['PROFILE', 'Platform']

Social Media DataFrame head after one-hot encoding:
```

	N	FOLLOWERS	POSTS	PROFILE_ЯGAG: Go Fun The World	PROFILE_<U+0001F451>	PROFILE_<U+0001F40B>julia mms garcia<U+0001F40B>	PROFILE_<U+0438>nokhina liza	PROFILE_<U+041C><U+0438><U+044B><U+0448><U+043E> <U+043E><U+041C><U+0435><U+0434><U+0432><U+0435><U+0434><U+046C>
0	1	539448845.0	7202.0	False	False	False	False	False
1	2	473084930.0	3338.0	False	False	False	False	False
2	3	384542528.0	6935.0	False	False	False	False	False
3	4	355790796.0	890.0	False	False	False	False	False
4	5	341579083.0	1828.0	False	False	False	False	False

```
5 rows × 471 columns

Social Media DataFrame info after encoding categorical features:
<class 'pandas.core.frame.DataFrame'>
Index: 465 entries, 0 to 499
Columns: 471 entries, N to Platform_YouTube
dtypes: bool(468), float64(2), int64(1)
memory usage: 227.1 KB
```

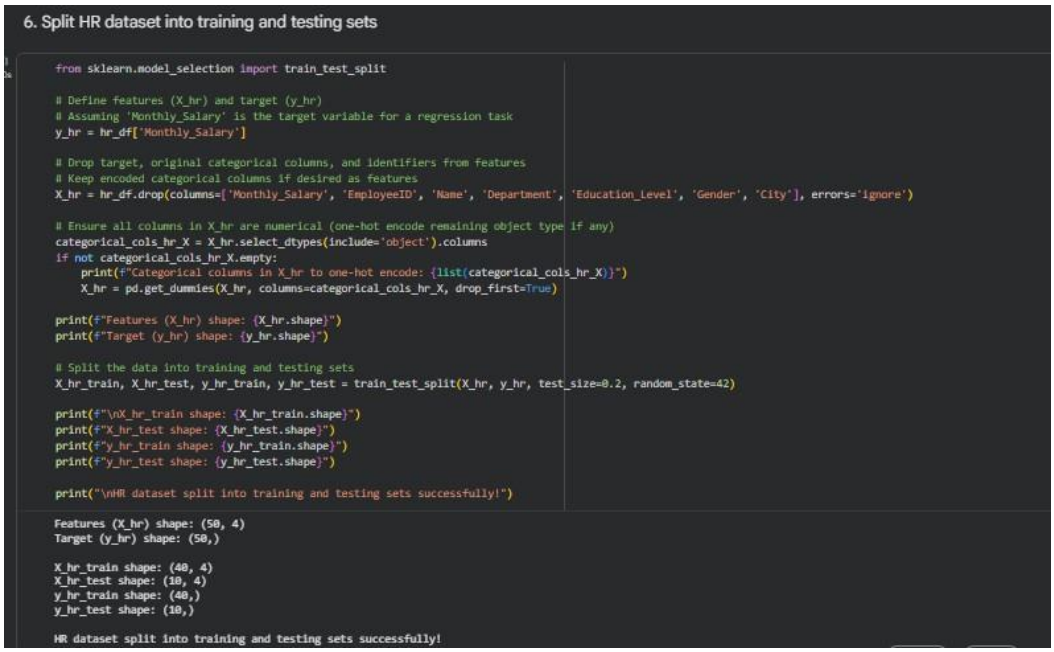
Cleened Dataset for Engagement Prediction Summary

```
print("Final preprocessed Social Media DataFrame head:")
display(social_df.head())
print("\nFinal preprocessed Social Media DataFrame info:")
social_df.info()
```

```
Final preprocessed Social Media DataFrame head:
```

	N	FOLLOWERS	POSTS	PROFILE_ЯGAG: Go Fun The World	PROFILE_<U+0001F451>	PROFILE_<U+0001F40B>julia mms garcia<U+0001F40B>	PROFILE_<U+0438>nokhina liza	PROFILE_<U+041C><U+0438><U+044B><U+0448><U+043E> <U+043E><U+041C><U+0435><U+0434><U+0432><U+0435><U+0434><U+046C>
0	1	539448845.0	7202.0	False	False	False	False	False
1	2	473084930.0	3338.0	False	False	False	False	False
2	3	384542528.0	6935.0	False	False	False	False	False
3	4	355790796.0	890.0	False	False	False	False	False
4	5	341579083.0	1828.0	False	False	False	False	False

```
5 rows × 471 columns

Final preprocessed Social Media DataFrame info:
<class 'pandas.core.frame.DataFrame'>
Index: 465 entries, 0 to 499
Columns: 471 entries, N to Platform_YouTube
dtypes: bool(468), float64(2), int64(1)
memory usage: 227.1 KB
```



Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

• Convert date column into datetime format.

• Handle missing values in temperature, humidity.

• Create new features (season, week_number).

• Normalize rainfall values.

Dataset link:

https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input

Expected Output:

• A processed dataset ready for forecasting models.

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load the weather forecast data
df = pd.read_csv('/content/weather_forecast_data.csv')
print("Original DataFrame head:")
display(df.head())
print("\nOriginal DataFrame info:")
df.info()

# 1. Convert date column into datetime format
# Assuming the date column is named 'Date'. Adjust if needed.
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
    # Drop rows where Date couldn't be parsed
    df.dropna(subset=['Date'], inplace=True)
    print("\nDate column converted to datetime format:")
    display(df.head())
else:
    print("Could not find a 'Date' column. Please check your CSV and adjust the column name if necessary.")

# 2. Handle missing values in temperature and humidity
missing_cols = ['Temperature', 'Humidity']
for col in missing_cols:
    if col in df.columns:
        if df[col].isnull().sum() > 0:
            # Impute numerical columns with the median
            df[col].fillna(df[col].median(), inplace=True)
            print(f"Missing values in '{col}' imputed with median.")
        else:
            print(f"No missing values found in '{col}'.")
    else:
        print(f"Column '{col}' not found in the DataFrame. Skipping missing value handling.")
print("\nMissing values after handling:")
display(df[missing_cols].isnull().sum())

# 3. Create new features (season, week_number)
if 'Date' in df.columns:
    df['Week_Number'] = df['Date'].dt.isocalendar().week.astype(int)

    # Define a function to determine the season
    def get_season(date):
        month = date.month
        if 3 <= month <= 5:
            return 'Spring'
        elif 6 <= month <= 8:
            return 'Summer'
        elif 9 <= month <= 11:
            return 'Autumn'
        else:
            return 'Winter'

    df['Season'] = df['Date'].apply(get_season)
    print("\nNew features 'Week_Number' and 'Season' created:")
    display(df.head())
else:
    print("Cannot create season and week_number features as 'Date' column is missing.")

# 4. Normalize rainfall values
if 'Rainfall' in df.columns:
    scaler = MinMaxScaler()
    df['Rainfall_Normalized'] = scaler.fit_transform(df[['Rainfall']])
    print("\n'Rainfall' values normalized:")
    display(df.head())
else:
    print("Could not find a 'Rainfall' column. Skipping normalization.")

# 5. Display processed dataset summary
print("\nProcessed dataset ready for forecasting models:")
```

```
Original DataFrame head:
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

```
Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Temperature  2500 non-null   float64
 1   Humidity     2500 non-null   float64
 2   Wind_Speed   2500 non-null   float64
 3   Cloud_Cover  2500 non-null   float64
 4   Pressure     2500 non-null   float64
 5   Rain         2500 non-null   object
dtypes: float64(5), object(1)
memory usage: 117.3+ KB
Could not find a 'Date' column. Please check your CSV and adjust the column name if necessary.
No missing values found in 'Temperature'.
No missing values found in 'Humidity'.

Missing values after handling:
```

	0
Temperature	0
Humidity	0

```
dtype: int64
Cannot create season and week_number features as 'Date' column is missing.
Could not find a 'Rainfall' column. Skipping normalization.

Processed dataset ready for forecasting models:
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 6 columns):
```

Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-

generated scripts.

Instructions:

• Handle missing values in rating, genre.

• Encode categorical variables like genre, language.

• Remove duplicate movie entries.

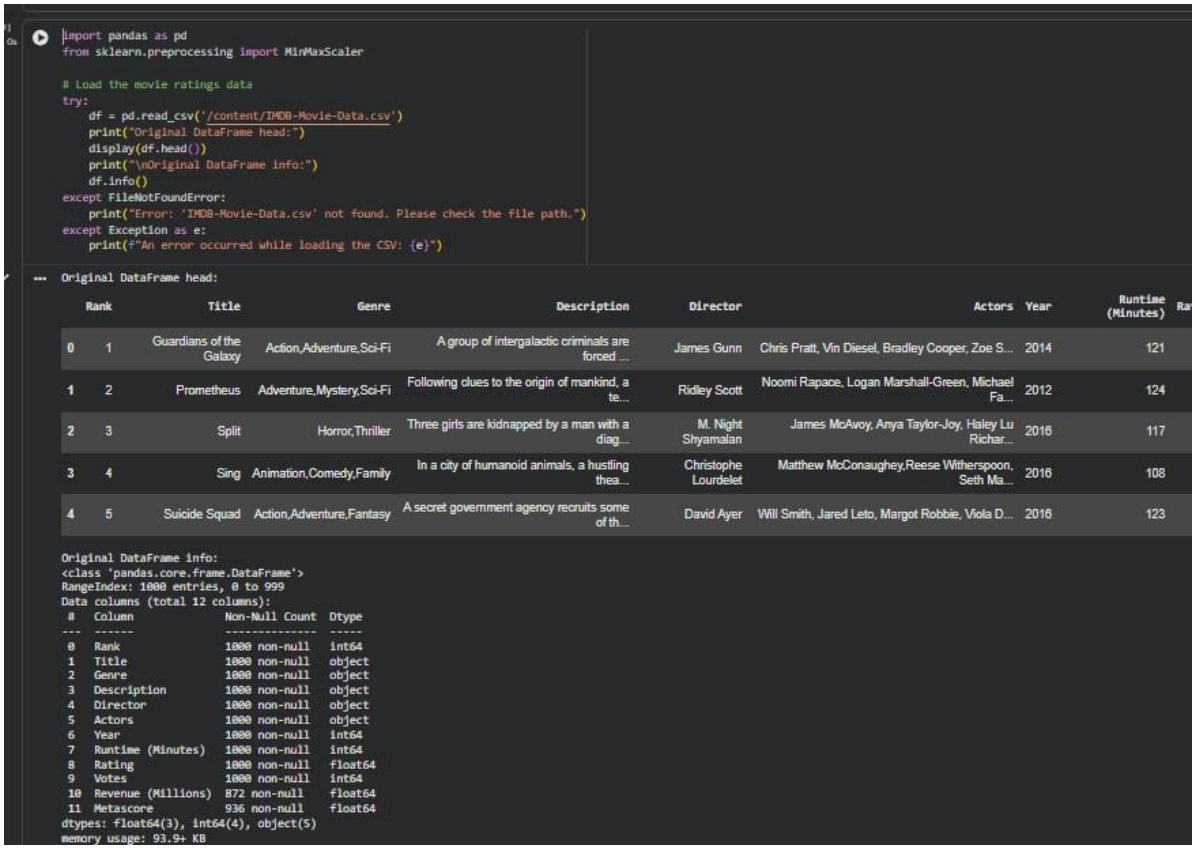
• Scale rating values between 0 and 1.

Dataset link:

https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data

Expected Output:

• A dataset suitable for recommendation systems.



```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load the movie ratings data
try:
    df = pd.read_csv('/content/IMDB-Movie-Data.csv')
    print("Original DataFrame head:")
    display(df.head())
    print("\nOriginal DataFrame info:")
    df.info()
except FileNotFoundError:
    print("Error: 'IMDB-Movie-Data.csv' not found. Please check the file path.")
except Exception as e:
    print(f"An error occurred while loading the CSV: {e}")
```

```
Original DataFrame head:
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Ra
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	

```
Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Rank                1000 non-null   int64
 1   Title               1000 non-null   object
 2   Genre               1000 non-null   object
 3   Description         1000 non-null   object
 4   Director            1000 non-null   object
 5   Actors              1000 non-null   object
 6   Year                1000 non-null   int64
 7   Runtime (Minutes)   1000 non-null   int64
 8   Rating              1000 non-null   float64
 9   Votes               1000 non-null   int64
 10  Revenue (Millions)  872 non-null    float64
 11  Metascore           936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

```python
print("Missing values before handling:")
display(df[['Rating', 'Genre']].isnull().sum())

# Handle missing values in 'Rating' (numerical) with median
if 'Rating' in df.columns:
    if df['Rating'].isnull().sum() > 0:
        df['Rating'].fillna(df['Rating'].median(), inplace=True)
        print("Missing values in 'Rating' imputed with median.")
    else:
        print("No missing values found in 'Rating'.")
else:
    print("Column 'Rating' not found. Skipping missing value handling for 'Rating'.")

# Handle missing values in 'Genre' (categorical) with mode
if 'Genre' in df.columns:
    if df['Genre'].isnull().sum() > 0:
        mode_genre = df['Genre'].mode()[0]
        df['Genre'].fillna(mode_genre, inplace=True)
        print(f"Missing values in 'Genre' imputed with mode: {mode_genre}.")
    else:
        print("No missing values found in 'Genre'.")
else:
    print("Column 'Genre' not found. Skipping missing value handling for 'Genre'.")

print("\nMissing values after handling:")
display(df[['Rating', 'Genre']].isnull().sum())
print("\nDataFrame head after handling missing values:")
display(df.head())
```

```
Missing values before handling:
```

	0
Rating	0
Genre	0

```
dtype: int64
No missing values found in 'Rating'.
No missing values found in 'Genre'.

Missing values after handling:
```

	0
Rating	0
Genre	0

```
dtype: int64

DataFrame head after handling missing values:
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	76.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	62.0
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	59.0
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	40.0

```python
initial_rows = df.shape[0]
if 'Title' in df.columns:
    df.drop_duplicates(subset=['Title'], inplace=True)
    print(f"Number of rows before removing duplicates: {initial_rows}")
    print(f"Number of rows after removing duplicates based on 'Title': {df.shape[0]}")
    print("\nDataFrame head after removing duplicates:")
    display(df.head())
else:
    print("Column 'Title' not found. Skipping duplicate removal.")
```

```
Number of rows before removing duplicates: 1000
Number of rows after removing duplicates based on 'Title': 999

DataFrame head after removing duplicates:
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	76.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	62.0
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	59.0
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	40.0

+ Code + Text

3. Encode categorical variables like 'Genre' and 'Language'

+ Code + Text

```python
categorical_cols_to_encode = ['Genre', 'Language']

for col in categorical_cols_to_encode:
    if col in df.columns:
        print(f"Encoding column: '{col}'")
        df = pd.get_dummies(df, columns=[col], drop_first=True)
    else:
        print(f"Column '{col}' not found. Skipping encoding for '{col}'.")

print("\nDataFrame head after one-hot encoding:")
display(df.head())
print("\nDataFrame info after encoding categorical features:")
df.info()
```

```
Encoding column: 'Genre'
Column 'Language' not found. Skipping encoding for 'Language'.

DataFrame head after one-hot encoding:
```

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Genre_Mystery,Romance,Thriller	Genre_Mystery,Sci-Fi,Thriller	Genre_Mystery,Thriller	Genre_Mystery,Thriller,Western
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	False	False	False	False
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	...	False	False	False	False
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	False	False	False	False

```python
if 'Rating' in df.columns:
    scaler = MinMaxScaler()
    df['Rating_Scaled'] = scaler.fit_transform(df[['Rating']])
    print("\'Rating\' values scaled between 0 and 1:")
    display(df[['Rating', 'Rating_Scaled']].head())
else:
    print("Column 'Rating' not found. Skipping scaling.")
```

```
'Rating' values scaled between 0 and 1:
```

	Rating	Rating_Scaled
0	8.1	0.873239
1	7.0	0.718310
2	7.3	0.760563
3	7.2	0.746479
4	6.2	0.605634

5. Processed Dataset Summary

```python
print("\nProcessed dataset ready for recommendation systems:")
display(df.head())
df.info()
```

```
Processed dataset ready for recommendation systems:
```

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Genre_Mystery,Sci-Fi,Thriller	Genre_Mystery,Thriller	Genre_Mystery,Thriller,Western	Genre_Romance,Sci-Fi	Genre_R
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	False	False	False	False	
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	...	False	False	False	False	
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	False	False	False	False	
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	...	False	False	False	False	
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	...	False	False	False	False	

```
5 rows × 218 columns
<class 'pandas.core.frame.DataFrame'>
Index: 999 entries, 0 to 999
Columns: 218 entries, Rank to Rating_Scaled
dtypes: bool(206), float64(4), int64(4), object(4)
```